Lesson 10 Assignment

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from sklearn import metrics
from sklearn.model_selection import train_test_split
```

1. Explore the Breast Cancer Data

Loading the breast cancer csv file and checking on statistical analysis

In [5]:
```
data = pd.read_csv("breastcancer.csv")
data.describe()
```

Out[5]:

	Age	BMI	Glucose	Insulin	HOMA	Leptin	Adiponectin	Resistin	MCP.1	Classificat
count	116.000000	116.000000	116.000000	116.000000	116.000000	116.000000	116.000000	116.000000	116.000000	116.000
mean	57.301724	27.582111	97.793103	10.012086	2.694988	26.615080	10.180874	14.725966	534.647000	1.551
std	16.112766	5.020136	22.525162	10.067768	3.642043	19.183294	6.843341	12.390646	345.912663	0.499
min	24.000000	18.370000	60.000000	2.432000	0.467409	4.311000	1.656020	3.210000	45.843000	1.000
25%	45.000000	22.973205	85.750000	4.359250	0.917966	12.313675	5.474283	6.881763	269.978250	1.000
50%	56.000000	27.662416	92.000000	5.924500	1.380939	20.271000	8.352692	10.827740	471.322500	2.000
75%	71.000000	31.241442	102.000000	11.189250	2.857787	37.378300	11.815970	17.755207	700.085000	2.000
max	89.000000	38.578759	201.000000	58.460000	25.050342	90.280000	38.040000	82.100000	1698.440000	2.000

Checking on any null values in the dataset

In [7]:
```
data.isna().sum()
```

Out[7]:
```
Age               0
BMI               0
Glucose           0
Insulin           0
HOMA              0
Leptin            0
Adiponectin       0
Resistin          0
MCP.1             0
Classification    0
dtype: int64
```

Checking on any duplicated rows in the dataset

In [9]:
```
data.duplicated().sum()
```

Out[9]:
```
0
```

Conclusion ➡️ The dataset has no null or duplicated values. We can proceed with the standardization and split of data into training and test sets.

Standardizing the data using MinMaxScaler (as the units are different for different columns of data) and splitting into train and test datasets

In [12]:
```
from sklearn import preprocessing
```

In [13]:
```
scaler = preprocessing.MinMaxScaler()
```

Standardizing data on all columns except Classification

In [15]:
```
data.iloc[:, data.columns != "Classification"] = scaler.fit_transform(data.iloc[:, data.columns != "Classification"])
data
```

Out[15]:

	Age	BMI	Glucose	Insulin	HOMA	Leptin	Adiponectin	Resistin	MCP.1	Classification
0	0.369231	0.253850	0.070922	0.004908	0.000000	0.052299	0.221152	0.060665	0.224659	1
1	0.907692	0.114826	0.226950	0.012190	0.009742	0.052726	0.103707	0.010826	0.255926	1
2	0.892308	0.235278	0.219858	0.036874	0.022058	0.158526	0.571021	0.076906	0.307912	1
3	0.676923	0.148328	0.120567	0.014171	0.005911	0.064811	0.151538	0.121131	0.533934	1
4	0.953846	0.135640	0.226950	0.019936	0.013748	0.027782	0.086940	0.093375	0.440565	1
...	...	...	...	...	...	...	...	...	...	...
111	0.323077	0.419620	0.226950	0.016028	0.011727	0.585897	0.287049	0.098238	0.134568	2
112	0.584615	0.419125	0.283688	0.037446	0.026441	0.094674	0.543206	0.052098	0.172043	2
113	0.630769	0.676934	0.262411	0.058863	0.036757	0.664996	0.573988	0.090252	0.162294	2
114	0.738462	0.357271	0.156028	0.006925	0.004189	0.240191	0.882091	0.000761	0.209741	2
115	0.953846	0.435950	0.553191	0.311951	0.256680	1.000000	0.342293	0.014451	0.026774	2

116 rows × 10 columns

In [16]:
```
X = data.drop("Classification", axis = 1)
y = data["Classification"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
```

2. Build and evaluate SVC models with different kernel function

We will start off by creating a dataframe to store metrics analysis of different types of kernel that would be used for SVC and store dummy values for the different metrics scores to begin with.

In [19]:
```
df_svc = pd.DataFrame()
df_svc["kernel"] = ["linear", "poly", "rbf", "sigmoid"]
df_svc[["accuracy_score", "precision_score", "recall_score", "f1_score"]] = 0
df_svc
```

Out[19]:

	kernel	accuracy_score	precision_score	recall_score	f1_score
0	linear	0	0	0	0
1	poly	0	0	0	0
2	rbf	0	0	0	0
3	sigmoid	0	0	0	0

We will now run the different kernel versions using default values of the hyperparameters to find out the best performing one using default hyperparameters

In [21]:

```
from sklearn import svm
for index, row in df_svc.iterrows():
    fn_svc = svm.SVC(kernel = row['kernel'], C = 1.0, degree = 3, gamma = 'scale', coef0 = 1.0)
    fn_svc.fit(X_train, y_train)
    y_pred = fn_svc.predict(X_test)
    accuracy_score = metrics.accuracy_score(y_test, y_pred)
    precision_score = metrics.precision_score(y_test, y_pred)
    recall_score = metrics.recall_score(y_test, y_pred)
    f1_score = metrics.f1_score (y_test, y_pred)
    df_svc[df_svc["kernel"] == row['kernel']] = [row['kernel'], accuracy_score, precision_score, recall_score, f1_sc

df_svc
```

```
C:\Users\HARIbol\anaconda3\envs\PSU_DAAN862\lib\site-packages\sklearn\metrics\_classification.py:1334: UndefinedMetri
cWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to
control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

Out[21]:

	kernel	accuracy_score	precision_score	recall_score	f1_score
0	linear	0.708333	0.777778	0.583333	0.666667
1	poly	0.875000	0.909091	0.833333	0.869565
2	rbf	0.833333	0.833333	0.833333	0.833333
3	sigmoid	0.500000	0.000000	0.000000	0.000000

Conclusion ➡️ We find that Sigmoid kernel performs the worst and is at chance level of accuracy (0.5) in this kernel comparison. The best performing kernel among the kernels is ploynomial kernel. We determine this based on recall score followed by F1 score to compare and find the best performing model under equivalent parameters.

We will now run GridSearchCV on SVC model to find the best model with the best hyperparameters

In [24]:
```
from sklearn.model_selection import GridSearchCV
```

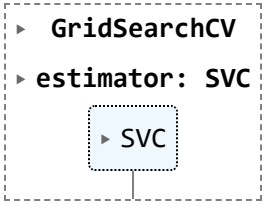
In [25]:
```
tuned_params = {"kernel" : ["linear", "poly", "rbf", "sigmoid"],
                "C" : [1.0, 10.0, 100.0, 1000.0],
                "degree" : [1, 2, 3, 4],
                "gamma" : [0.1, 1.0, 2.0, 3.0, 4.0],
                "coef0" : [0.1, 1.0, 2.0, 3.0, 4.0]}

scoring_keys = ["accuracy", "precision", "recall", "f1"]
scoring = {key:None for key in scoring_keys}
scoring
```

Out[25]:
```
{'accuracy': None, 'precision': None, 'recall': None, 'f1': None}
```

In [26]:
```
best_svc = svm.SVC()
grid_search = GridSearchCV(best_svc, tuned_params, cv = 10, scoring = scoring, refit = "recall")
grid_search.fit(X_train, y_train)
```

Out[26]:



In [27]:
```
grid_search.best_params_
```

Out[27]:
```
{'C': 100.0, 'coef0': 0.1, 'degree': 1, 'gamma': 0.1, 'kernel': 'rbf'}
```

In [28]:
```
best_svc_model = grid_search.best_estimator_
y_pred = best_svc_model.predict(X_test)
accuracy_score = metrics.accuracy_score(y_test, y_pred)
precision_score = metrics.precision_score(y_test, y_pred)
recall_score = metrics.recall_score(y_test, y_pred)
```

```
f1_score = metrics.f1_score (y_test, y_pred)
df_svc_best = pd.Series({"kernel": "best_model " + grid_search.best_params_['kernel'],
                         "accuracy_score": accuracy_score,
                         "precision_score": precision_score,
                         "recall_score": recall_score,
                         "f1_score": f1_score})

df_svc = pd.concat([df_svc, df_svc_best.to_frame().T], ignore_index = True)

df_svc
```

Out[28]:

	kernel	accuracy_score	precision_score	recall_score	f1_score
0	linear	0.708333	0.777778	0.583333	0.666667
1	poly	0.875	0.909091	0.833333	0.869565
2	rbf	0.833333	0.833333	0.833333	0.833333
3	sigmoid	0.5	0.0	0.0	0.0
4	best_model rbf	0.916667	0.857143	1.0	0.923077

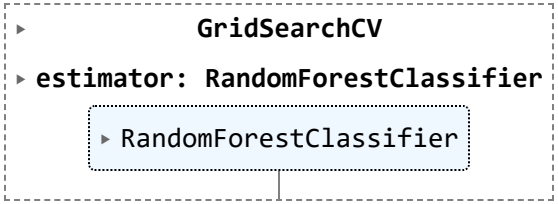
Conclusion ➡️ The best model that we could determine is a RBF kernel with C value as 100, coeff0 value as 0.1, degree as 1 and gamma as 0.1

3. Find the best n_estimator for Random Forest model

In [31]:
```
n_estimator = list(range (5, 101, 5))
min_samples_split = list(range(2,6,3))
min_samples_leaf = list(range(1,6,2))
tuned_params = {"n_estimators" : n_estimator,
                "criterion" : ["gini", "entropy", "log_loss"],
                "min_samples_split" : min_samples_split,
                "min_samples_leaf" : min_samples_leaf,
                "max_features" : ["sqrt", "log2"]}
```

In [32]:
```
from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier()
grid_search = GridSearchCV(rfc, tuned_params, cv = 5, scoring = scoring, refit = "recall")
grid_search.fit(X_train, y_train)
```

Out[32]:



In [33]:
```
grid_search.best_params_
```

Out[33]:
```
{'criterion': 'gini',
 'max_features': 'log2',
 'min_samples_leaf': 5,
 'min_samples_split': 2,
 'n_estimators': 30}
```

In [34]:
```
best_rfc_model = grid_search.best_estimator_
y_pred = best_rfc_model.predict(X_test)
accuracy_score = metrics.accuracy_score(y_test, y_pred)
precision_score = metrics.precision_score(y_test, y_pred)
recall_score = metrics.recall_score(y_test, y_pred)
f1_score = metrics.f1_score (y_test, y_pred)
best_rfc = pd.Series({"n_estimator": grid_search.best_params_['n_estimators'],
                      "accuracy_score": accuracy_score,
                      "precision_score": precision_score,
                      "recall_score": recall_score,
                      "f1_score": f1_score})
best_rfc
```

Out[34]:
```
n_estimator        30.000000
accuracy_score      0.958333
precision_score     1.000000
recall_score        0.916667
f1_score            0.956522
dtype: float64
```

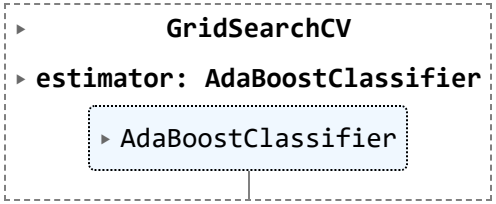
Conclusion ➡️ The best n_estimator for Random Forest Classification is 30 with criterion as gini, max_features as log2, min_samples_leaf as 5 and min_samples_split 2.

4. Find the best n_estimator for Adaboost model

In [37]:
```
n_estimator = list(range (5, 101, 5))
tuned_params = {"n_estimators" : n_estimator}
```

In [38]:
```
from sklearn.ensemble import AdaBoostClassifier
abc = AdaBoostClassifier()
grid_search = GridSearchCV(abc, tuned_params, cv = 5, scoring = scoring, refit = "recall")
grid_search.fit(X_train, y_train)
```

Out[38]:



In [39]:
```
grid_search.best_params_
```

Out[39]:
```
{'n_estimators': 55}
```

In [40]:
```
best_abc_model = grid_search.best_estimator_
y_pred = best_abc_model.predict(X_test)
accuracy_score = metrics.accuracy_score(y_test, y_pred)
precision_score = metrics.precision_score(y_test, y_pred)
recall_score = metrics.recall_score(y_test, y_pred)
f1_score = metrics.f1_score (y_test, y_pred)
best_abc = pd.Series({"n_estimator": grid_search.best_params_['n_estimators'],
                      "accuracy_score": accuracy_score,
                      "precision_score": precision_score,
                      "recall_score": recall_score,
                      "f1_score": f1_score})
best_abc
```

Out[40]:
```
n_estimator        55.000000
accuracy_score      0.791667
precision_score     0.733333
recall_score        0.916667
f1_score            0.814815
dtype: float64
```

Conclusion ➡️ The best n_estimator for Adaboost Classifier is 55.

End of Assignment